数据资产档案 · 附件 5

人机协同过程数据治理规范（草案）V1.1

人机协同创作长程推理链数据集

新型数据品类的生产、治理、验证与合规资产化实践

主体：皇清华（苏州过程思维数据科技有限公司）

2026年10月

规范精华｜解决什么问题

本规范面向以会话导出文件为原始形态的人机协同创作、决策和设计过程数据，目标是把“留存下来的对话”治理为**可核验、可追溯、可进入后续登记与流通流程的数据资产**。

核心方法

方法	核心要求
六步治理管线	采集导出→容错解析→轮次化→归属判定→守恒切割→质检统计
逐轮结构	一组人类输入＋紧随的一组AI回复；含推理链者为三元组，不含者为两元组
N＋2归属	N个作品类＋元活动类＋泛化兜底类；允许多归属，提到作品名不等于实质归属
多方判定与仲裁	机器初筛、独立双判、机器比对、分歧分类、人工仲裁、随机抽检、校准重跑
切割守恒与溯源	切口双侧留缓冲，片段记录源窗口与截取范围；条数守恒可机器校验
质量与口径管理	机器自检、交叉验证、人工亲审；中间产物留档，口径变迁单独记录

适用边界

- 适用于JSON等会话导出数据，不直接覆盖实时流式采集和音视频原始信号治理。
- 规范覆盖治理层，不替代定价、登记申报、授权范围等交易与权属文件。
- 平台输出权益与商用条款核查是治理前置条件，规范条文本身不替代相应核查证据。
- 外部中文文献仅作内部研究对照，不进入数据产品、不流通、不商用。

目录与复现指引

部分	内容	页数	总页码
前置	规范精华、目录与复现指引	3页	1—3
第一部分	总则、术语与适用范围	3页	4—6
第二部分	治理管线、归属体系与仲裁机制	4页	7—10
第三部分	切割、命名、质量、交付与合规存证	4页	11—14
第四部分	参数表与B1—B5通用参考脚本	8页	15—22

样例能够证明什么

样例	实测结果	证明边界
最小样例：12窗、2来源包、N＝2	B1：60轮／三元组45；B2：检出预设C3与A类翻转；B3：12／12幂等改名；B4：14＝14；B5：14项缺失0	证明五个脚本可运行并产生预期输出，不用于复算全量477窗、18,744轮
真实样例《微尘纪元》：3窗	62轮全三元组；B1、B5及MD5可复核	展示真实数据与治理产物形态；未触发仲裁和切割机制

复现入口

附件5.1根目录README提供包结构，通用脚本README提供参数说明，两个样例包分别提供目录、预期输出和复核方法。当前成果定位为**可复现治理脚本及样例**，不是具备统一部署入口的完整软件平台。

> **页眉说明：**每页同时提供全册总页码与本部分页码；第4页起为规范全文及附录。

人机协同过程数据治理规范（草案）V1.1

> 本规范是”人机协同过程数据”品类的通用治理模板（草案），面向任何采用”人与 AI 长程会话协同创作”方式生产的过程数据集。全文参数化：所有与具体数据集相关的取值（作品数量、窗口数量、目录路径、平台名称等）均以【】占位，使用前按本数据集实际填写（附录 A 附最小填写示例）。本规范的首次完整实施记录见配套文件《治理工程实施档案》。
>
> 修订记录：V1.0（2026-10）首版，原名《人机协同过程数据治理规范（通用模板）》；V1.1（2026-10）依第三方评审意见修订并定名《人机协同过程数据治理规范（草案）V1.1》——管线表改为分步条目式排版、附录 A 增加最小填写示例、附录 B 五个参考脚本经最小样例实测运行通过、新增口径手册与案例库要求、合规检查清单、规范边界声明。

苏州过程思维数据科技有限公司　（2026 年 10 月）

第一章　总则

1.1 目的

规范人机协同过程数据（“人类输入＋AI 推理链＋AI 回答”逐轮对齐的长程会话全过程记录）从原始采集到可交易数据资产的治理全流程，使治理成果满足：可核验（全部统计数字可从原始文件独立实计复核）、可追溯（任一片段可回溯到原始窗口）、可流通（结构、命名、权属文件满足登记与交易要求）。

1.2 适用范围

- **适用**：以会话导出文件（JSON 等）为原始形态的人机协同创作/决策/设计过程数据集的治理；
- **不适用**：实时流式采集、多模态原始信号（音视频）的治理；涉及第三方个人信息的语料须先完成合规审查再参照本规范执行。

1.3 治理前提条件

治理开始前，输入数据应满足：

1. 原始会话文件为平台原生导出，未经人工改写；
2. 每条消息含角色标识（人类/AI）与时间戳（缺时间戳的平台须另行补证）；
3. AI 消息中推理链与正式回答可区分（含独立字段或可识别标记）；
4. 定稿成果文件可得，且能与创作过程建立对应关系；
5. 采集所涉平台的输出内容权属与商用条款已完成合规核查（前置条件，属合规文件而非本规范内容）。

1.4 保真原则（最高原则）

正文零删改。对导出缺陷仅做序列化层面的还原（换行、转义符），不改写内容；缺失轮次以缺测标记如实标注，不补写、不合成、不清洗掉”难看”的内容。

1.5 规范边界

本规范覆盖治理层：采集、解析、轮次化、归属、切割、质检、统计、交付结构、存证互证。**不覆盖**交易层——定价、登记申报、授权范围、披露边界等由交易与权属文件另行规定；本规范的交付物（含 MD5、时间戳、总表体系）是交易层文件的输入。

第二章　术语与定义

术语	定义
窗口	一次完整会话的原始导出文件，治理的最小单元
消息	窗口内单条人类输入或 AI 回复记录
消息组	连续同角色消息并为一组
轮	一组人类输入＋紧随的一组 AI 回复
三元组	AI 回复含推理链的轮次：（人类输入，AI 推理链，AI 回答）
两元组	AI 回复不含推理链的轮次
推理链（思维链）	AI 给出正式回答前的可视化逐步推理内容
缺测标记	对缺失内容（如无推理链、无回复）的如实标注，不补写
归属	窗口（或片段）与作品/类别之间的对应关系
多归属	一个窗口同时归属多个类别，归档时复制入每个归属文件夹
切割	将混合主题窗口按内容实质拆分为多个单主题片段
守恒	切割前后消息条数满足固定等式，可机器校验
溯源注释	片段头部记录源窗口 ID、截取范围、切割

术语	定义
	依据的说明
定稿	协同创作的最终成果文件
过程包	按归属类别组织的窗口/片段归档文件夹
口径手册	记录本数据集全部判定口径、边界案例与仲裁先例的累积文档（见第 5.9 节）

第三章　治理管线（六步）

治理按六步管线执行，每步统一定义目的、输入、输出、校验点与通过标准。

第①步　采集与导出——目的：全过程留痕。输入：平台原生会话。输出：原始导出文件【N_WIN】个。校验点：文件可解析性。通过标准：全部文件成功解析，解析失败的修复并记录。

第②步　容错解析——目的：还原导出缺陷。输入：原始导出文件。输出：规范化 JSON＋全文渲染文本。校验点：字数抽核。通过标准：修复仅限序列化层，正文零删改。

第③步　轮次化——目的：建立标准轮次结构。输入：规范化 JSON。输出：逐轮统计明细。校验点：轮次连续性。通过标准：消息组配对规则一致执行，无回复组如实计数。

第④步　归属判定——目的：每窗归入 N+2 类别。输入：全文渲染文本。输出：判定总表。校验点：多方一致性。通过标准：见第五章仲裁机制闭环。

第⑤步　守恒切割——目的：混合窗拆分。输入：切割标记窗口。输出：片段文件＋溯源注释。校验点：条数守恒公式。通过标准：见第六章守恒公式逐窗通过。

第⑥步　质检与统计——目的：三级质检＋落盘。输入：全部治理产物。输出：统计表组＋校验报告。校验点：MD5/实计复核。通过标准：哈希零不一致，统计可独立复算。

第四章　归属体系通用范式（N+2 结构）

4.1 结构

归属类别数＝N＋2，随数据集规模动态伸缩：

- **W1…W_N　作品类**：每份定稿一类，共 N 类（N＝本数据集定稿作品数【待填】）。窗口可多归属；
- **M　元活动类**：主线围绕”数据集本身”的活动（研究探讨、实验验证、工程操作、推广交易、检索收录、未来展望），而非任何具体作品的文本创作；
- **G　泛化兜底类**：与全部作品及数据集主题均无直接关联的内容（清单外创作项目、工具问答、闲谈等）。

4.2 判定规则（通用）

1. **实质/仅提及二分**：对作品有实质创作贡献（写作、打磨、战略分析、模拟推演、求认证评估等）判”实质”；仅出现作品名而无实质内容判”仅提及”；
2. **提到≠归属**：窗口提到某作品名称，不等于归属该作品，按内容主线判定；
3. **多归属允许**：归属可组合（以+连接）；一窗整体只入一个主文件夹的，主归属＝证据最强的类别；多归属窗口归档时复制入每个归属文件夹；
4. **判贡献不看时间看内容**：定稿完成后的延伸探讨，如对作品仍有实质贡献，仍归作品类；

5. **元活动类与作品类边界**：围绕某作品文本本身的认证、打磨、推广归该作品类；围绕”数据集”这一产品本身的上架、定价、参赛、合规、检索归 M 类；两者皆有实质内容则双归属。

4.3 判定输出六字段

每个窗口的判定记录须含：**归属｜创作行为（实质/仅提及/无关）｜状态｜有效片段（整窗/部分，部分即需切割）｜置信度｜理由**。

第五章　多方判定与仲裁机制（通用）

归属判定采用”机器初筛→独立双判→机器比对→分歧分类→人工仲裁→随机抽检→校准重跑”闭环：

1. **代码初筛**：关键词打分（如 TF-IDF）为每窗产出归属候选 1–3 个，仅供判定参考，不作定案；
2. **首轮判定**：判定方分【N_BATCH】批通读窗口全文，按第 4.3 节六字段产出首轮判定；
3. **盲判复检**：复检方在**不看首轮判定**的条件下独立产出第二意见；
4. **机器比对**：逐窗比对两轮判定，输出分歧清单；
5. **分歧分类**：
   - A 类＝仅创作行为翻转（归属一致，实质/仅提及互翻）；
   - B 类＝仅 M/G 类差异；
   - C 类＝作品归属实质分歧，细分为：C1 复检扩充（首轮集合⊆复检集合）、C2 作品全转 M/G 类、C3 真冲突（互有增减）、C4 复检缩减；

6. **人工仲裁**：C3＋C4 逐窗亲审定锤；低置信/待定窗全部亲审；A 类按预设口径批量校准；B 类/C1/C2 按规则采纳复检；
7. **随机抽检**：以**固定随机种子**抽取不低于【1/9】比例的窗口人工核验；发现跨批次口径不一致的，统一校准后重跑总表；
8. **口径优先级**：数据生产者校准口径为最高优先级，全程统一，并全文留痕；
9. **口径手册与案例库**：治理过程中应同步累积本数据集的判定口径手册与案例库——每条校准口径、每个边界案例、每例仲裁理由均编号留痕（如 R1、R2……），后续批次判定与复检须先读口径手册再执行，防止跨批次口径漂移。口径手册随数据集一并归档交付。

第六章　守恒切割规则（通用）

1. 切割按话题转折点粗切，不强制精确到每一轮；
2. **双侧缓冲**：切口前后两段各多留【B＝3】条消息作为上下文（每切口重叠 2B 条），保证各片段可读连贯；
3. **溯源注释**：每个片段文件头部注明——源窗口 ID、原文件名、截取范围（起止时间/消息序号）、切割依据；
4. **碎段并段**：短于【MIN_FRAG＝8】条消息的碎片并入相邻段，归属取并集；
5. **守恒公式（机器校验）**：

   片段消息条数合计 ＝ 原窗口消息条数 ＋ 2B × 切口数

   多归属产生的片段副本只校验内容一致，不重复计入合计；
6. 无法满足守恒公式的窗口，须逐索引核验覆盖连续性并出具书面说明，方可判定通过。

第七章　命名规范（通用）

7.1 窗口文件名模板

【序号】_【起始日期】_【归属简写】_【关键词】.json

- 序号补零至三位；日期 YYYYMMDD；关键词剔除非法字符并限长；
- 平台默认导出名一律替换为规范名；同包内防重名（追加序号）；
- 改名必须产出**改名对照表**（窗口 ID／原文件名／新文件名／归属／创作行为／状态），全程可回查。

7.2 过程包内二次命名模板

【归属码】_【窗口 ID】_【阶段】_【状态】_【日期】.json

阶段按窗口在该归属内的时间序位划分：唯一窗口为”全期”；否则按序位比例划分初期/中期/末期/最终（分界比例【0.25/0.6/0.9】）。

第八章　质量控制体系

1. **三级质检**：机器自检（脚本断言：行数、ID 连续、归属合法、无待定残留）＋交叉验证（独立复检比对）＋人工亲审（仲裁与抽检）；
2. **内容摘要写实**：每窗摘要须描述真实内容，不得残留裁定流程行话；随机抽审与原文吻合为通过；
3. **MD5 全量比对**：归档副本与源文件逐一哈希比对，零不一致为通过；数据包整体计算 MD5 写入存证与交易文件；
4. **独立实计复核**：全部对外统计数字须能从原始文件由独立脚本重新实计得出，与统计表零偏差。

第九章　交付物模板

交付物	内容
来源原始包	按平台/来源分包的全部原始窗口，与源目录哈希一致
过程包	N＋2 个归属文件夹：整窗副本＋切割片段＋溯源注释＋本包统计小表；无过程窗的作品类放定稿与说明签
定稿包	全部定稿文件
总匹配表	一行一窗：窗口 ID／文件名／起止时间／消息数／字数／初判候选／最终归属／创作行为／状态／有效片段／置信度／裁定来源／理由／处置
逐轮统计明细	一行一轮：窗口 ID／轮次／来源包／人类字数／推理链字数／回复字数／轮次总字数／是否三元组／人类消息数／AI 消息数
统计汇总	总量、推理链含缺、归属分布、行为/状态/置信度分布、月度分布、切割统计
校验记录	仲裁记录、抽检结果、守恒校验、哈希比对结论

第十章　合规与伦理边界

1. 缺失如实标注，不补写、不合成；
2. 私人内容（闲谈等）独立归入泛化兜底类，与可流通内容分域管理；
3. 治理中用作内部对照的外部文献仅作研究对照，不进入数据产品、不流通、不商用；
4. 对话主体仅为数据生产者与 AI 的，应在交付文件中声明无第三方个人信息。

合规检查清单（治理启动前逐项核查，留痕备查）：

#	检查项	通过标准
1	来源平台协议核查	各来源平台现行协议对输出内容的权属与商用条款已逐条核查并留存截图/原文
2	第三方个人信息识别	窗口内容不含第三方可识别个人信息；含私人内容的已归入泛化兜底类分域管理
3	外部文献隔离	治理中引用的外部文献仅作内部对照，有书面隔离纪律与来源清单备查
4	权属链条	时间戳存证、登记文件、权属转让安排与数据包 MD5 互证一致
5	缺测与剔除声明	缺链窗口、剔除窗口均有如实声明，剔除理由与剔除前后口径对照留档

第十一章　可复核性与存证

1. 全部中间产物（判定表、分歧表、仲裁记录、对照表）归档留存，不随终表定稿删除；
2. 治理完成后对数据包整体计算 MD5，并与时间戳存证、登记文件互证；
3. 发生口径变更（如来源剔除、补采合并）时，保留原口径档案，出具口径变迁说明，新旧口径数字均可独立复算。

附录 A　参数表（使用前填写）

参数	含义	本数据集取值	填写示例
N	定稿作品数（作品类 W1–W_N）	【待填】	2
N_WIN	窗口总数	【待填】	12
N_BATCH	判定/复检批次数	【待填】	2
PACKAGES	来源分包清单	【待填】	包 A、包 B
B	切割缓冲条数（单侧）	3	3
MIN_FRAG	碎段并段阈值（条）	8	8
AUDIT_RATIO	抽检比例	≥1/9	2/12
随机种子	抽检批次抽样种	【待填】	20261028

参数	含义	本数据集取值	填写示例
RANDOM_SEED	子		

附录 B　通用参考脚本（参数化）

以下脚本为治理管线关键环节的参考实现，逻辑通用；参数区按本数据集实际填写后即可运行。脚本假定输入 JSON 为消息数组（或含 messages 字段的对象），每条消息含 role 与 contents 数组，contents 元素以 type 区分 text（正文）/thinking（推理链）——若来源平台导出结构不同，须先在第②步容错解析阶段归一化为该结构。**五个脚本已按附录 A"最小填写示例"参数在一个 12 窗口、2 来源包、N=2 的最小样例数据集上实测运行通过**：B.1 输出 60 轮（三元组 45，占 75.0%）；B.2 正确检出预设的 C3 真冲突与 A 类行为翻转各 1 例（全一致批次零误报）；B.3 改名 12/12 且幂等；B.4 切割守恒校验 14=14 通过；B.5 归档完整性 14 个校验项零缺失。

B.1 轮次化与逐轮统计

```
# -*- coding: utf-8 -*-
# 轮次化与逐轮统计:
# 轮=连续人类消息组+紧随的AI回复组;三元组=AI回复含thinking块
import json, glob, os
import pandas as pd

# ===== 参数区（按本数据集填写）=====
BASE = '【工作目录】'              # 原始窗口JSON根目录
PACKAGES = ['【来源包1】', '【来源包2】']
EXPECT_WINDOWS = 0                # 预期窗口总数（0=不校验）
OUT_CSV = '【输出路径】/逐轮统计明细.csv'

rows, no_reply, role_other = [], 0, set()
files = []
for pk in PACKAGES:
    files += glob.glob(f'{BASE}/{pk}/*.json')
if EXPECT_WINDOWS:
    assert len(files) == EXPECT_WINDOWS, len(files)
```

```
for fp in sorted(files):
    wid = int(os.path.basename(fp).split('_')[0])
    d = json.load(open(fp))
    msgs = d if isinstance(d, list) else d.get('messages', d)
    groups = []
    for m in msgs:
        r = m.get('role')
        if r not in ('user', 'assistant'):
            role_other.add(r)
        texts, thinks = '', ''
        for c in m.get('contents', []):
            if not isinstance(c, dict):
                continue
            if c.get('type') == 'text':
                texts += c.get('content', '') or ''
            if c.get('type') == 'thinking':
                thinks += c.get('content', '') or ''
        if groups and groups[-1][0] == r:
            groups[-1][1] += texts
            groups[-1][2] += thinks
            groups[-1][3] += 1
        else:
            groups.append([r, texts, thinks, 1])
    i, rnd = 0, 0
    while i < len(groups):
        g = groups[i]
        if g[0] != 'user':
            i += 1
            continue
        if i + 1 < len(groups) and groups[i+1][0] == 'assistant':
            a = groups[i+1]
            rnd += 1
            rows.append({
                '窗口 ID': wid, '轮次': rnd,
                '人类字数': len(g[1]),
                '推理链字数': len(a[2]),
                '回复字数': len(a[1]),
                '轮次总字数': len(g[1]) + len(a[2]) + len(a[1]),
                '三元组': '三元组' if a[2] else '两元组',
                '人类消息数': g[3], 'AI 消息数': a[3]})
            i += 2
        else:
            no_reply += g[3]
            i += 1

df = pd.DataFrame(rows)
```

```
df.to_csv(OUT_CSV, index=False, encoding='utf-8-sig')
total = len(df)
tri = int((df['三元组'] == '三元组').sum())
print(f'总轮次: {total:,} | 三元组: {tri:,} '
      f'({tri/total*100:.1f}%) | 两元组: {total-tri:,}')
print(f'无回复消息: {no_reply} | 异常角色: {role_other or "无"}')
```

B.2 双判比对与分歧分类

```
# -*- coding: utf-8 -*-
# 首轮判定 vs 盲判复检：逐窗比对 + 分歧分类
# A=行为翻转 B=仅元/泛化类差异 C=作品归属分歧
# C 类细分：C1 复检扩充 C2 作品转元/泛化类 C3 真冲突 C4 复检缩减
import pandas as pd, re

# ===== 参数区 =====
N_WORKS = 0                  # 定稿作品数 N
META, GENERAL = 'M', 'G'     # 元活动类/泛化兜底类代号
ORIG_CSV = '【首轮判定总表.csv】'
RECHECK_PATTERN = '【复检批次目录】/批次{n}.csv'
N_BATCH = 0                  # 复检批次数
OUT_CSV = '【输出】/分歧分类.csv'

WSET = {f'W{i:02d}' for i in range(1, N_WORKS + 1)}
WORKS = WSET | {META, GENERAL}

def split_set(s):
    parts = re.split(r'[+十、]', str(s))
    return set(w.strip() for w in parts
               if w.strip() and w.strip() not in ('待定', '无'))

orig = pd.read_csv(ORIG_CSV, dtype=str).fillna('')
orig['窗口 ID'] = orig['窗口 ID'].astype(int)
rows = []
for n in range(1, N_BATCH + 1):
    df = pd.read_csv(RECHECK_PATTERN.format(n=n), dtype=str)
    df = df.fillna('')
    df['窗口 ID'] = df['窗口 ID'].astype(int)
    df.columns = ['窗口 ID', '归属 2', '创作行为 2', '状态 2',
                  '有效片段 2', '置信度 2', '理由 2']
    for _, r in orig.merge(df, on='窗口 ID').iterrows():
        s1, s2 = split_set(r['归属']), split_set(r['归属 2'])
        if s1 == s2 and r['创作行为'] == r['创作行为 2']:
            continue
        w1, w2 = s1 & WSET, s2 & WSET
```

```
        if w1 == w2 and (s1 - WSET) == (s2 - WSET):
            typ, sub = 'A 行为翻转', ''
        elif w1 == w2:
            typ, sub = 'B 仅元泛化类差异', ''
        else:
            typ = 'C 作品分歧'
            if not w2:
                sub = 'C2 作品转元泛化类'
            elif not w1:
                sub = 'C3 真冲突'
            elif w1 <= w2:
                sub = 'C1 复检扩充'
            elif w2 < w1:
                sub = 'C4 复检缩减'
            else:
                sub = 'C3 真冲突'
        rows.append(dict(
            批次=n, 窗口 ID=r['窗口 ID'], 类型=typ, 子类=sub,
            原归属=r['归属'], 复检归属=r['归属 2'],
            原行为=r['创作行为'], 复检行为=r['创作行为 2'],
            原置信=r['置信度'], 复检置信=r['置信度 2'],
            复检理由=r['理由 2'][:400], 原理由=r['理由'][:400]))

out = pd.DataFrame(rows)
out.to_csv(OUT_CSV, index=False, encoding='utf-8-sig')
print(out['类型'].value_counts())
# C3/C4 → 人工逐窗仲裁；A 类按预设口径批量校准；
# B/C1/C2 按规则采纳复检
```

B.3 窗口规范化改名

```
# -*- coding: utf-8 -*-
# 窗口规范化改名：序号_日期_归属简写_关键词.json
# 产出改名对照表（可回查、幂等）
import pandas as pd, re, os

# ===== 参数区 =====
BASE = '【工作目录】'
FINAL_TABLE = '【归属终表.csv】'
# 归属简写映射，按本数据集补全：
SHORT = {'W01': '【作品 1 简写】', 'M': '【元活动类简写】',
         'G': '【泛化类简写】'}

ILLEGAL = r'[\\/:*?"<>|《》 （） (), ,。 .、 \s]+'
```

```
def clean(s, n=22):
    s = re.sub(ILLEGAL, '', str(s))
    return s[:n] if s else ''

t = pd.read_csv(FINAL_TABLE, dtype=str).fillna('')
t['窗口 ID'] = t['窗口 ID'].astype(int)
used, rows = {}, []
for _, r in t.iterrows():
    wid = r['窗口 ID']
    date = str(r['起始时间'])[:10].replace('-', '')
    works = [w.strip() for w in re.split(r'[+＋]', r['最终归属'])]
    kw = ''.join(SHORT.get(w, w) for w in works[:2])
    kw = clean(kw, 16) or '窗口'
    new = f"{wid:03d}_{date}_{'_'.join(works)}_{kw}.json"
    key = (r['所在文件包'], new)
    if key in used:
        used[key] += 1
        new = new[:-5] + f"_{used[key]}.json"
    else:
        used[key] = 0
    rows.append(dict(
        窗口 ID=wid, 所在文件包=r['所在文件包'],
        原文件名=r['原文件名'], 新文件名=new,
        最终归属=r['最终归属'], 创作行为=r['创作行为'],
        状态=r['状态']))

m = pd.DataFrame(rows)
ok, miss = 0, []
for _, r in m.iterrows():
    d = f"{BASE}/{r['所在文件包']}"
    p_old = f"{d}/{r['原文件名']}"
    p_new = f"{d}/{r['新文件名']}"
    if os.path.exists(p_old):
        os.rename(p_old, p_new)
        ok += 1
    elif os.path.exists(p_new):
        ok += 1          # 幂等：已改过
    else:
        miss.append((r['窗口 ID'], r['原文件名']))
m.to_csv(f'{BASE}/改名对照表.csv', index=False,
         encoding='utf-8-sig')
print(f'改名成功 {ok}/{len(m)}，缺失 {len(miss)}')
```

B.4 切割守恒校验

```python
# -*- coding: utf-8 -*-
# 切割守恒校验：片段条数合计 = 原条数 + 2B × 切口数
# 多归属副本只校验内容一致，不重复计入合计
import json, glob, os, re

# ===== 参数区 =====
BASE = '【原始窗口根目录】'
PROC = '【过程包目录】'
CUT = [0]           # 需切割窗口 ID 清单
B = 3               # 单侧缓冲条数

def orig_count(wid):
    pats = glob.glob(f'{BASE}/*/{wid:03d}_*.json')
    if not pats:
        return None
    d = json.load(open(pats[0]))
    return len(d if isinstance(d, list) else d.get('messages', d))

def load_msgs(fp):
    d = json.load(open(fp))
    return d if isinstance(d, list) else d.get('messages', d)

bad, total_unique = [], 0
for wid in CUT:
    frags = glob.glob(f'{PROC}/*/*_{wid:03d}_片段*.json')
    uniq = {}
    for fp in frags:
        m = re.search(r'_片段([A-Z])_', os.path.basename(fp))
        letter = m.group(1) if m else '?'
        msgs = load_msgs(fp)
        if letter in uniq:
            if len(uniq[letter]) != len(msgs):
                bad.append(f'{wid:03d}片段{letter}: 副本条数不一致')
            continue
        uniq[letter] = msgs
    if not uniq:
        bad.append(f'{wid:03d}: 无片段!')
        continue
    n_cuts = len(uniq) - 1
    got = sum(len(v) for v in uniq.values())
    expect = orig_count(wid) + 2 * B * n_cuts
    total_unique += len(uniq)
    status = 'OK' if got == expect else f'差{got - expect:+d}'
    print(f'{wid:03d}: {len(uniq)}片段 '
```

```
          f'合计{got} 期望{expect} {status}')
    if got != expect:
        bad.append(f'{wid:03d}: {status}')

print(f'唯一片段总数: {total_unique} | '
      f'守恒通过: {len(CUT) - len(bad)}/{len(CUT)}')
```

B.5 归档完整性校验

```
# -*- coding: utf-8 -*-
# 归档完整性校验: 每窗 x 每个归属码,
# 在对应归属文件夹中须有副本（整窗或片段）
import os, re
import pandas as pd

# ===== 参数区 =====
PROC = '【过程包目录】'
FINAL_TABLE = '【归属终表.csv】'
# 归属码 -> 文件夹名映射, 按本数据集补全:
WFOLDER = {'W01': '【W01 文件夹名】',
           'M': '【元活动类文件夹】',
           'G': '【泛化类文件夹】'}

PAT = re.compile(r'^(?:(?:W\d{2}|[MG])\+?)+_(\d{3})_')

idx = {}
for folder in os.listdir(PROC):
    fp = f'{PROC}/{folder}'
    if not os.path.isdir(fp):
        continue
    d = {}
    for fn in os.listdir(fp):
        m = PAT.match(fn)
        if m:
            d.setdefault(int(m.group(1)), []).append(fn)
    idx[folder] = d

t = pd.read_csv(FINAL_TABLE, dtype=str).fillna('')
t['窗口 ID'] = t['窗口 ID'].astype(int)
missing, checked = [], 0
for _, r in t.iterrows():
    attrs = [x.strip() for x in r['最终归属'].split('+')]
    for a in attrs:
        checked += 1
        folder = WFOLDER[a]
        if r['窗口 ID'] not in idx.get(folder, {}):
```

```
            missing.append(f"{r['窗口 ID']:03d} 缺 {a}")

print(f'归属-窗校验项: {checked} | 缺失: {len(missing)}')
for x in missing[:50]:
    print(' !', x)
```

本规范文本与附录脚本可自由用于同类人机协同过程数据的治理实践；引用时请注明出处。规范版本 V1.1，首次完整实施例见《治理工程实施档案——人机协同创作长程推理链数据集》。